Currently in qlego i was running optimization passes experiments... however the main problem that I'm facing is how to conduct experiments by combining various passes it is unlike layout/routing where u can replace an entite pass here passes could act complementarily to each other.... Could u suggest some methology by which I could combine iptimization passes ofc I could consider all the passes at once but how to select subset of them?

Yes — optimization passes are much harder than layout/routing because they are **not usually mutually exclusive alternatives**. They behave more like:

- enabling passes
- cleanup passes
- aggregation/analysis passes
- target-aware rewriting passes
- iterative local improvement passes

So the right methodology is usually **not "pick one pass from a menu"**, but:

1. Treat optimization as a staged pipeline

Instead of selecting arbitrary subsets from all passes, divide passes into **roles**.

A practical grouping is:

A. Canonicalization / normalization

These make circuits easier for later passes to recognize.

Examples:

- basis translation / gate normalization
- parameter simplification
- unrolling to a controlled gate vocabulary
- commutation analysis / collecting blocks

B. Structure exposure

These expose patterns for other passes.

Examples:

- `Collect2qBlocks`
- `CollectMultiQBlocks`
- `CollectCliffords`
- commutation-based reordering
- block formation

C. Local rewrite / simplification

These actually reduce cost.

Examples:

- `Optimize1qGates`
- `Optimize1qGateDecomposition`
- `CommutativeCancellation`
- `OptimizeSwapBeforeMeasure`
- `RemoveDiagonalGatesBeforeMeasure`
- Clifford simplification

D. Target-aware resynthesis / rebasing

These matter only relative to backend basis/coupling/error model.

Examples:

- rebase to target basis
- 2q block synthesis into native entanglers
- Clifford+T or backend-native decomposition

E. Cleanup

Examples:

- remove identities
- remove final resets
- remove dead measurements or diagonal-before-measure gates

This already gives you a much better search space:
you search over **stage choices**, not raw arbitrary subsets.

---

2. Use a “skeleton + optional modules” design

Define a base skeleton that is always present, for example:

```
normalize -> expose structure -> local simplify -> target-aware synthesize -> cleanup
```

Then for each stage, choose among a few alternatives or skip.

For instance:

- Normalize:
  - none
  - basis translator
  - translate parametrized gates + basis translator
- Structure exposure:
  - none
  - collect 2q blocks
  - collect cliffords
  - commutation analysis + collect blocks
- Local simplify:
  - `Optimize1qGates`
  - `CommutativeCancellation`
  - both
  - iterative loop of both
- Target-aware:
  - none
  - backend rebase
  - 2q resynthesis
  - Clifford optimization
- Cleanup:
  - fixed cleanup set

This is far better than brute-forcing all subsets of 20+ passes.

---

3. Select subsets by compatibility graph, not by raw power set

You should build a **pass interaction graph**.

For each pass, annotate:

- **preconditions**: what must already be true
- **postconditions**: what structure it creates
- **destructive effects**: what structure it may destroy
- **target dependence**: whether it needs backend/basis info

- **repeatability**: whether rerunning can still help

Then you can combine passes only if they are logically compatible.

Example:

- `Collect2qBlocks` is useful **before** block synthesis
- `BasisTranslator` too early may destroy higher-level structure
- `CommutativeCancellation` may expose more 1q merges
- `Optimize1qGates` and commutation/cancellation often work well iteratively
- some rebasing passes should happen late, because early rebasing may prevent later algebraic simplifications

So instead of subset selection over all passes, select subsets that form a valid DAG of dependencies.

---

4. Search over pass bundles, not individual passes

A very useful idea is to define a small number of **bundles/macros**.

For example:

Bundle 1: 1q cleanup

- `Optimize1qGates`
- `Optimize1qGateDecomposition`

Bundle 2: commutation cleanup

- `CommutationAnalysis`
- `CommutativeCancellation`

Bundle 3: block resynthesis

- `Collect2qBlocks`
- backend-aware 2q synthesis / decomposition

Bundle 4: measurement cleanup

- `OptimizeSwapBeforeMeasure`
- `RemoveDiagonalGatesBeforeMeasure`
- `ResetAfterMeasureSimplification`

Bundle 5: Clifford cleanup

- `CollectCliffords`
- `OptimizeCliffords`
- maybe `OptimizeCliffordT`

Then experiment on **bundle subsets**, not pass subsets.
That shrinks the search space dramatically and is more semantically meaningful.

---

5. Use forward selection, not exhaustive subset enumeration

A very practical experimental methodology is **greedy forward selection**:

Step 1

Start with a baseline pipeline.

Step 2

Evaluate adding each candidate pass or bundle individually.

Step 3

Keep the one that gives the best average gain across benchmark circuits.

Step 4

Repeat from the new pipeline until no statistically meaningful improvement occurs.

This gives you a subset in a principled way.

You can also do:

- **backward elimination**: start with many passes, remove one at a time
- **beam search**: keep top-k partial pipelines
- **sequential model-based optimization** if the space grows large

Forward selection is usually the best first methodology.

---

6. Measure marginal contribution conditionally

A pass may look useless alone but help after another pass.
So you should not only compute:

- gain of pass P from baseline

but also:

- gain of P conditioned on pipeline S

That is exactly why staged or greedy selection works better.

Useful quantities:

- **standalone improvement**
- **conditional improvement after selected pipeline**
- **interaction term**:

$$\Delta(P_i, P_j) = f(S + \{P_i, P_j\}) - f(S + \{P_i\}) - f(S + \{P_j\}) + f(S)$$

If this is positive, the passes are complementary.

This lets you identify pass pairs or bundles that should stay together.

---

7. Use iterative fixed-point loops for complementary passes

Some passes only become useful after others fire.

Typical example:

- commute gates
- cancel gates
- merge 1q runs
- expose new commuting opportunities
- cancel again

So instead of applying each once, define **micro-loops** like:

```
[CommutationAnalysis -> CommutativeCancellation -> Optimize1qGates] x N
```

Stop when:

- no metric improves, or
- max iterations reached

This is much more realistic than selecting these as isolated single applications.

---

8. Evaluate on multiple metrics, not one scalar only

Optimization passes may trade off:

- total gate count
- 2q gate count
- depth

- critical path duration
- estimated fidelity
- T-count / Clifford+T cost
- runtime of compilation

So define a score like:

$$\text{Score} = w_1 \cdot \Delta(2q) + w_2 \cdot \Delta(\text{depth}) + w_3 \cdot \Delta(\text{duration}) + w_4 \cdot \Delta(\text{fidelity})$$

Or keep a Pareto frontier.

For qlego, this is especially important because different SDKs optimize for different notions of "better".

---

9. Stratify benchmark circuits before selecting subsets

A single subset may not be best for all circuits.

You should partition circuits by features such as:

- width
- depth
- fraction of 2q gates
- commutation richness
- Clifford ratio
- parameterized vs non-parameterized
- measurement-heavy / dynamic-circuit presence
- native-basis closeness

Then learn/select bundles per cluster.

This is much more meaningful than one global best subset.

---

10. A concrete methodology for qlego

I would suggest this exact workflow:

Phase 1: Pass taxonomy

For every optimization pass in each SDK, annotate:

- category
- prerequisites
- whether analysis-only or transform
- target-aware or not
- likely companions
- cheap/expensive
- repeatable or once-only

Phase 2: Build bundles

Create 5–10 reusable bundles like:

- 1q simplify
- commutation cleanup
- Clifford cleanup
- 2q block resynthesis
- measurement cleanup
- basis normalization
- final cleanup

Phase 3: Define a skeleton

Example:

```
normalize -> [optional exposure] -> [optional simplify loop] -> [optional target-aware
synthesis] -> cleanup
```

Phase 4: Search at bundle level

Use:

- greedy forward selection
- optionally beam search with width 3–5

Phase 5: Interaction study

For top pipelines, compute pairwise interaction gains to identify complementary bundles.

Phase 6: Per-cluster specialization

Train/select different optimization recipes for different circuit families.

---

11. What not to do

I would avoid these initially:

A. Raw power set over all passes

Too large, uninterpretable, and many subsets are nonsense.

B. Comparing passes as if they are alternatives

That works for layout/routing, not optimization.

C. Single-shot application only

Many optimization passes are effective only in loops.

D. Early rebasing everywhere

It may destroy structure that later passes could exploit.

---

12. Best experimental framing for a paper/project

You can present it as:

> We model optimization-pass composition as a staged combinatorial selection problem rather than a flat alternative-choice problem. Passes are grouped into semantically meaningful bundles, and candidate pipelines are built by selecting bundle instances within a fixed optimization skeleton. We then use greedy/beam search to identify subsets with positive marginal contribution under multi-objective cost metrics.

That framing is solid and publishable enough for methodology.

---

13. My strongest recommendation

For qlego, the cleanest first version is:

Search unit:

bundle, not individual pass

Pipeline form:

fixed skeleton with optional stages

Search algorithm:

greedy forward selection + small iterative loops inside stages

Evaluation:

multi-objective and benchmark-stratified

That gives you:

- manageable search space
- interpretable results
- fair cross-framework comparison
- room later for RL/AutoML phase ordering

A simple example search space could be:

- Normalize: {none, basis translate}
- Exposure: {none, collect2q, collectCliffords}
- Simplify loop: {none, 1q only, commutation+cancellation, both}
- Target-aware: {none, rebase, block synth}
- Cleanup: {fixed}

This is already a strong experimental design.

If you want, I can next help you design a **formal metadata schema for optimization passes in qlego** so subset selection can be automated cleanly.

Can u suggest of any paper which have considered this approach?

Thought for 17s

Yes — a few papers are quite close to what you are describing, though not always in exactly the same "bundle compatible passes into subsets" language.

The **closest quantum paper for subset selection** is **COMPASS: Compiler Pass Selection For Improving Fidelity Of NISQ Applications**. It explicitly argues that applying *all* available passes is not necessarily best, and instead searches for an **optimal subset of passes** for a given circuit and device. Their scalable variant, **E-COMPASS**, reduces the search space by **splitting passes into smaller groups**, searching within each group, and then combining the best group-level choices. That is very close to your idea of not treating optimization as one giant flat power set. arXiv

If you want a **quantum paper on composing sequences of optimization passes adaptively**, then the strongest current match is **Reinforcement Learning for Adaptive Composition of Quantum Circuit Optimisation Passes** (2026). It trains an RL agent that repeatedly chooses the **next optimization pass** based on the circuit state, so the pass sequence is built incrementally rather than fixed in advance. That paper is more about **sequence construction** than bundle selection, but it supports your point that optimization passes are complementary and should be chosen conditionally, not as one-shot alternatives. arXiv

An earlier and important quantum reference is **Compiler Optimization for Quantum Computing Using Reinforcement Learning** by Quetschlich, Burgholzer, and Wille. Their framework combines passes from **Qiskit and TKET** in a single compilation flow and learns optimized pass sequences rather than relying on one default compiler pipeline. This is not exactly "subset grouping," but it is very relevant as prior art for **cross-framework pass composition** and **phase-order optimization** in quantum compilation. arXiv +1

For the exact methodological idea of **grouping dependent or mutually beneficial passes into subsequences/bundles**, the clearest paper I found is actually from the **classical compiler** literature: **Efficient compiler optimization by modeling passes dependence**. It explicitly says that some passes have **mutual benefits or dependencies**, and proposes organizing related passes into **sub-sequences**, so the search becomes a search over combinations of subsequences rather than over all individual passes. That is almost exactly the methodology I suggested for qlego. Springer

Another useful classical reference is **Learning Compiler Pass Orders using Coreset and Normalized Value Prediction**. Instead of searching over arbitrary individual-pass sequences, it first identifies a **small**

set of representative pass sequences and then learns to pick among them. Conceptually, this is close to replacing raw pass-level search with a smaller space of **meaningful candidate recipes**. Proceedings of M...

A more recent classical angle is **Leveraging Compilation Statistics for Compiler Phase Ordering (CITROEN)**, which focuses on **order and combination of passes** and uses statistics to reject low-value sequences quickly. It is less directly about bundling, but it is relevant if you want a justification for using **guided search over a reduced structured space** instead of brute force. White Rose Rese...

So if you want the most defensible prior-work story for your qlego methodology, I would cite it like this:

1. **Quantum subset selection:** COMPASS / E-COMPASS. arXiv
2. **Quantum adaptive sequence composition:** Quetschlich et al. and Mills et al. arXiv +1
3. **Pass-dependency / bundle methodology:** DCO from classical compilers. Springer

So the honest answer is: **yes, there is prior work nearby**, and **the single paper closest to your "group complementary passes and search over groups" idea is the classical DCO paper**, while **the closest quantum paper is COMPASS/E-COMPASS**. arXiv +1

I can also turn this into a **related work paragraph for your paper/proposal** with 5–8 citations in proper academic style.

Sources

so the bundles that u suggested .... is there any paper which have suggested that bundle?

I'm checking whether those specific bundle categories appear explicitly in prior work, versus being a synthesis from compiler practice and pass APIs.

Thought for 19s

Not really as an **explicit named bundle taxonomy** in the quantum literature.

What I suggested — things like:

- canonicalization / normalization
- structure exposure
- local simplification
- target-aware resynthesis
- cleanup

is mostly a **methodological synthesis**, not a bundle list I can point to in one paper. I do **not** know of a quantum paper that says "use exactly these five optimization bundles" in that form. The closest support is indirect. Qiskit itself organizes compilation as fixed **stages** in a `StagedPassManager`, and its docs distinguish an `init` stage for more abstract work from an `optimization` stage for low-level, hardware-aware optimization, which supports the general idea of staged organization rather than flat pass selection. qiskit.org +1

For the specific pass pairings I suggested, the evidence is mostly at the **pass-function level**, not the "bundle" level. For example, Qiskit documents `Optimize1qGatesDecomposition` as combining chains of 1-qubit gates, `CommutativeCancellation` as canceling gates using commutation relations, and the pass catalog includes passes such as `Collect2qBlocks`, `CollectCliffords`, `OptimizeCliffords`, `OptimizeSwapBeforeMeasure`, and `RemoveDiagonalGatesBeforeMeasure`. That supports grouping them by role — block collection, commutation cleanup, Clifford cleanup, measurement cleanup — but the grouping itself is an interpretation, not a published taxonomy. qiskit.org +2

The closest paper to the **idea of grouping dependent passes into meaningful sub-sequences** is actually from classical compilation: **"Efficient compiler optimization by modeling passes dependence."** It builds an optimization dependence graph and generates optimization **sub-sequences**, which is very much the spirit behind the bundle idea. But again, it does not propose the exact quantum bundles I listed. Springer

So the honest answer is:

- **No**, I do not know a paper that proposes **those exact bundles** as written.
- **Yes**, there is support for the broader methodology:
  - staged compilation pipelines in Qiskit, qiskit.org +1
  - pass-specific roles in the Qiskit pass catalog, qiskit.org +2
  - dependency-based grouping into sub-sequences in classical compiler literature. Springer

So if you write this up, the safest wording is something like:

> "We introduce a role-based bundling of optimization passes for qlego, inspired by staged compiler pipelines and prior work on pass-dependence modeling, rather than adopting a bundle taxonomy directly from prior quantum-compilation literature." qiskit.org +2

That would be accurate.

I can also help you turn these bundles into a **defensible related-work paragraph** so it sounds academically careful rather than ad hoc.

Sources

optimization_experiment_results

qlego-qiskit_PresetOptimizationPass
qlego-qiskit_Optimize1qGatesPass
qlego-qiskit_Optimize1qGateDecompositionPass
qlego-qiskit_Optimize1qGateSimpleCommutationPass
qlego-qiskit_CommutativeCancellationPass
qlego-qiskit_CommutationAnalysisPass
qlego-qiskit_CommutativeOptimizationPass
qlego-qiskit_OptimizeCliffordsPass
qlego-qiskit_OptimizeCliffordTPass
qlego-qiskit_CollectCliffordsPass
qlego-qiskit_Collect2qBlocksPass
qlego-qiskit_CollectMultiQBlocksPass
qlego-qiskit_RemoveIdentityEquivalentPass
qlego-qiskit_OptimizeSwapBeforeMeasurePass
qlego-qiskit_RemoveDiagonalGatesBeforeMeasurePass
qlego-qiskit_RemoveFinalResetPass
qlego-tket_SquashTK1Pass
qlego-tket_SquashRzPhasedXPass
qlego-tket_DecomposeSingleQubitsTK1Pass
qlego-tket_FullPeepholeOptimizePass
qlego-tket_CliffordSimpPass
qlego-tket_PeepholeOptimize2QPass
qlego-tket_KAKDecompositionPass
qlego-tket_RemoveRedundanciesPass
qlego-tket_RemovePhaseOpsPass
qlego-tket_DelayMeasuresPass
qlego-tket_SimplifyMeasuresPass
qlego-tket_RemoveDiscardedPass
qlego-tket_SynthesiseTketPass
qlego-bqskit_GroupSingleQuditGatePassPass
qlego-bqskit_ScanningGateRemovalPassPass
qlego-bqskit_IterativeScanningGateRemovalPassPass
qlego-bqskit_TreeScanningGateRemovalPassPass
qlego-bqskit_QuickPartitionerPass
qlego-bqskit_LEAPSynthesisPassPass
qlego-bqskit_QSearchSynthesisPassPass

qlego-qiskit_PresetOptimizationPass
qlego-qiskit_Optimize1qGatesPass
qlego-qiskit_Optimize1qGateDecompositionPass
qlego-qiskit_Optimize1qGateSimpleCommutationPass
qlego-qiskit_CommutativeCancellationPass
qlego-qiskit_CommutationAnalysisPass
qlego-qiskit_CommutativeOptimizationPass
qlego-qiskit_OptimizeCliffordsPass
qlego-qiskit_OptimizeCliffordTPass
qlego-qiskit_CollectCliffordsPass
qlego-qiskit_Collect2qBlocksPass
qlego-qiskit_CollectMultiQBlocksPass
qlego-qiskit_RemoveIdentityEquivalentPass
qlego-qiskit_OptimizeSwapBeforeMeasurePass
qlego-qiskit_RemoveDiagonalGatesBeforeMeasurePass
qlego-qiskit_RemoveFinalResetPass
qlego-tket_SquashTK1Pass
qlego-tket_SquashRzPhasedXPass
qlego-tket_DecomposeSingleQubitsTK1Pass
qlego-tket_FullPeepholeOptimizePass
qlego-tket_CliffordSimpPass
qlego-tket_PeepholeOptimize2QPass
qlego-tket_KAKDecompositionPass
qlego-tket_RemoveRedundanciesPass
qlego-tket_RemovePhaseOpsPass
qlego-tket_DelayMeasuresPass
qlego-tket_SimplifyMeasuresPass
qlego-tket_RemoveDiscardedPass
qlego-tket_SynthesiseTketPass
qlego-bqskit_GroupSingleQuditGatePassPass

qlego-bqskit_ScanningGateRemovalPassPass
qlego-bqskit_IterativeScanningGateRemovalPassPass
qlego-bqskit_TreeScanningGateRemovalPassPass
qlego-bqskit_QuickPartitionerPass
qlego-bqskit_LEAPSynthesisPassPass
qlego-bqskit_QSearchSynthesisPassPass

this is an exhaustive list of passes from qiskit, tket and bqskit... how should I bundle them?

Thought for a couple of seconds

Yes — with this list, I would **not** bundle by framework first.
I would bundle by **optimization role**, then map each framework's passes into those roles.

That way qlego can compare:

- same-role bundles across frameworks
- mixed-framework pipelines
- complementary bundles instead of random pass subsets

A good bundling for your list would be this.

---

1. Single-qubit simplification bundle

These passes mostly compress or canonicalize 1q gate strings.

Qiskit

- `Optimize1qGatesPass`
- `Optimize1qGateDecompositionPass`
- `Optimize1qGateSimpleCommutationPass`

TKET

- `SquashTK1Pass`

ChatGPT Share

- `DecomposeSingleQubitsTK1Pass`

BQSKit

- `GroupSingleQuditGatePassPass`

Meaning

This is your **1q normalization / compression** bundle.

Notes

- `DecomposeSingleQubitsTK1Pass` is slightly different since it rewrites into TK1 form rather than only reducing gates, but it belongs here.
- In experiments, this bundle is usually safe to run early and sometimes again late.

---

2. Commutation and algebraic simplification bundle

These passes use commutation structure or algebraic local rewrites.

Qiskit

- `CommutativeCancellationPass`
- `CommutationAnalysisPass`
- `CommutativeOptimizationPass`

TKET

- `FullPeepholeOptimizePass`
- `RemoveRedundanciesPass`
- `RemovePhaseOpsPass`

BQSKit

- `ScanningGateRemovalPassPass`
- `IterativeScanningGateRemovalPassPass`
- `TreeScanningGateRemovalPassPass`

Meaning

This is your **local simplification / cancellation / peephole** bundle.

Notes

- `CommutationAnalysisPass` is analysis-only, so I would mark it as a **support pass** inside the bundle, not a standalone optimization pass.
- `FullPeepholeOptimizePass` is broad and may overlap with other bundles; still, practically it belongs here.
- BQSKit scanning removal passes fit nicely here because they are local elimination-style optimizations.

---

3. Clifford-aware simplification bundle

These exploit Clifford structure specifically.

Qiskit

- `OptimizeCliffordsPass`
- `OptimizeCliffordTPass`
- `CollectCliffordsPass`

TKET

- `CliffordSimpPass`

BQSKit

- none obvious from this list

Meaning

This is your **Clifford structure exploitation** bundle.

Notes

- `CollectCliffordsPass` is again a structure-exposure/helper pass.
- `OptimizeCliffordTPass` is a bit more specialized than plain Clifford simplification, so you may want:
  - bundle 3A: Clifford simplification
  - bundle 3B: Clifford+T simplification

If your benchmarks are not Clifford/T-heavy, keep this bundle optional.

---

4. Multi-qubit block collection / structure exposure bundle

These passes do not always directly optimize, but they expose bigger units for later synthesis.

Qiskit

- `Collect2qBlocksPass`
- `CollectMultiQBlocksPass`

TKET

- no exact direct analog from this list

BQSKit

- `QuickPartitionerPass`

Meaning

This is your **structure exposure / partitioning** bundle.

Notes

This bundle is important because it often helps **later synthesis bundles**.
I would almost never evaluate these passes alone; evaluate them together with synthesis/resynthesis.

---

5. Multi-qubit synthesis / resynthesis bundle

These replace blocks by better decompositions.

Qiskit

- no explicit strong synthesis pass in this list except maybe preset optimization indirectly

TKET

- `PeepholeOptimize2QPass`
- `KAKDecompositionPass`
- `SynthesiseTketPass`

BQSKit

- `LEAPSynthesisPassPass`
- `QSearchSynthesisPassPass`

Meaning

This is your **block resynthesis / target-aware rewrite** bundle.

Notes

This is one of the most important bundles in your whole set.
Also:

- `KAKDecompositionPass` is really a 2q synthesis/decomposition pass
- `PeepholeOptimize2QPass` sits between simplification and resynthesis, but I would place it here because it targets 2q structure
- `SynthesiseTketPass` is broad; depending on your implementation, you may separate it as a “global synthesis” bundle

---

6. Measurement and terminal cleanup bundle

These passes optimize around measurement/reset/end-of-circuit structure.

Qiskit

- `OptimizeSwapBeforeMeasurePass`
- `RemoveDiagonalGatesBeforeMeasurePass`
- `RemoveFinalResetPass`

TKET

- `DelayMeasuresPass`
- `SimplifyMeasuresPass`
- `RemoveDiscardedPass`

BQSKit

- none obvious

Meaning

This is your **measurement / terminal cleanup** bundle.

Notes

This bundle is especially useful if your circuits contain measurement, reset, or terminal swaps.
For pure unitary benchmarks, you can skip it.

---

7. Identity / dead-code cleanup bundle

These are pure cleanup passes.

Qiskit

- `RemoveIdentityEquivalentPass`

TKET

- `RemoveRedundanciesPass`
- `RemoveDiscardedPass`

BQSKit

- scanning removal passes partly overlap

Meaning

This is your **cheap cleanup** bundle.

Notes

You can either keep this as a separate final bundle or merge it into bundle 2 and bundle 6.
Practically, I would often treat it as a **fixed always-run cleanup bundle**.

---

8. Preset / macro bundle

Qiskit

- `PresetOptimizationPass`

Meaning

This is not really a primitive bundle member. It is a **macro pipeline**.

Recommendation

Do **not** mix it inside the same search space as primitive passes at first.

Use it as:

- baseline
- upper/lower comparison point
- ablation reference

Because preset optimization may already include effects overlapping with many other Qiskit passes, making attribution messy.

My recommended final bundle set

If you want something clean and experimentally manageable, use these **6 main bundles**:

B1. Single-qubit simplification

- Qiskit: `Optimize1qGates`, `Optimize1qGateDecomposition`, `Optimize1qGateSimpleCommutation`
- TKET: `SquashTK1`, `SquashRzPhasedX`, `DecomposeSingleQubitsTK1`
- BQSKit: `GroupSingleQuditGatePass`

B2. Local algebraic / commutation simplification

- Qiskit: `CommutationAnalysis`, `CommutativeCancellation`, `CommutativeOptimization`
- TKET: `FullPeepholeOptimize`, `RemoveRedundancies`, `RemovePhaseOps`
- BQSKit: `ScanningGateRemoval`, `IterativeScanningGateRemoval`, `TreeScanningGateRemoval`

B3. Clifford simplification

- Qiskit: `CollectCliffords`, `OptimizeCliffords`, `OptimizeCliffordT`
- TKET: `CliffordSimp`

B4. Block formation / partitioning

- Qiskit: `Collect2qBlocks`, `CollectMultiQBlocks`
- BQSKit: `QuickPartitioner`

B5. Block synthesis / resynthesis

- TKET: `PeepholeOptimize2Q`, `KAKDecomposition`, `SynthesiseTket`
- BQSKit: `LEAPSynthesis`, `QSearchSynthesis`

B6. Measurement / terminal cleanup

- Qiskit: `OptimizeSwapBeforeMeasure`, `RemoveDiagonalGatesBeforeMeasure`, `RemoveFinalReset`
- TKET: `DelayMeasures`, `SimplifyMeasures`, `RemoveDiscarded`

And optionally:

B7. Final cheap cleanup

- `RemoveIdentityEquivalent`
- `RemoveRedundancies`
- `RemoveDiscarded`

Even better: classify passes as helper vs transformer

Inside each bundle, mark each pass as one of:

Helper / analysis / collector

These expose structure.
Examples:

- `CommutationAnalysisPass`
- `CollectCliffordsPass`
- `Collect2qBlocksPass`
- `CollectMultiQBlocksPass`
- `QuickPartitionerPass`

Transformer

These actually change the circuit.
Examples:

- `CommutativeCancellationPass`
- `Optimize1qGatesPass`
- `KAKDecompositionPass`
- `LEAPSynthesisPassPass`

This matters because **helper-only bundles should usually not be tested alone**.

Suggested pipeline skeleton

Now, instead of arbitrary subsets, use a skeleton like:

$$[B1] \to [B2] \to [B3 \text{ optional}] \to [B4 + B5] \to [B6] \to [B7]$$

That is:

1. 1q simplification
2. local cancellation/peephole
3. Clifford simplification if relevant
4. block collection + resynthesis
5. measurement cleanup

6. final dead-code cleanup

This is much better than treating all passes equally.

Important constraints for experiments

Rule 1: never separate B4 and B5 too much

`Collect2qBlocks`, `CollectMultiQBlocks`, and `QuickPartitioner` are most meaningful when followed by synthesis.

So test:

- `B4+B5`
  not just `B4`.

Rule 2: keep preset optimization outside the primitive search

Use `PresetOptimizationPass` only as:

- baseline
- comparison pipeline
- ablation reference

Rule 3: do not force all frameworks to fill every bundle

For example BQSKit may dominate B5, while Qiskit dominates B3/B6.

That is okay. Your goal is not symmetry; your goal is meaningful role-based composition.

Rule 4: allow repeated B1/B2

A good pipeline may be:

$$B1 \to B2 \to B4 + B5 \to B1 \to B2 \to B6$$

because synthesis often creates new 1q simplification opportunities.

Smallest practical search space

If you want a very manageable first experiment, search over only:

- B1 on/off
- B2 on/off
- B3 on/off
- B4+B5 choice:
  - none
  - TKET resynthesis
  - BQSKit resynthesis
- B6 on/off

That is already strong and interpretable.

My strongest recommendation for qlego

Use these 5 top-level experimental bundles:

1. **1Q_SIMPLIFY**
2. **LOCAL_CANCEL**
3. **CLIFFORD_SIMPLIFY**
4. **BLOCK_RESYNTH** = structure exposure + synthesis together
5. **MEASURE_CLEANUP**

Then keep:

- `FINAL_CLEANUP` always on

- `PresetOptimizationPass` as external baseline only

That is the cleanest setup.

If you want, I can next turn this into a **Python dictionary / metadata schema for qlego**, like:

```python
PASS_BUNDLES = {
    "1Q_SIMPLIFY": [...],
    "LOCAL_CANCEL": [...],
    ...
}
```

with per-pass fields such as `role`, `framework`, `helper_only`, `repeatable`, and `requires_measurements`.

so within a bundle I can choose a particular pass right?

Yes — that is the right way to do it.

Think of a bundle as a **role bucket**, not "run all passes in the bucket."

So for a bundle, you can choose:

- **none**
- **one pass**
- **one small recipe** inside that bundle
- later, maybe **multiple passes** if they are naturally paired

For example, in `1Q_SIMPLIFY`, the choice could be:

- none
- Qiskit `Optimize1qGates`
- Qiskit `Optimize1qGateDecomposition`
- TKET `SquashTK1`
- TKET `SquashRzPhasedX`
- BQSKit `GroupSingleQuditGatePass`

That is much cleaner than saying all of them must run together.

But I would add one important refinement:

Two-level selection

Level 1: choose whether a bundle is active

Example:

- `1Q_SIMPLIFY` : on/off
- `LOCAL_CANCEL` : on/off
- `BLOCK_RESYNTH` : on/off

Level 2: choose the implementation inside the bundle

Example:

- if `1Q_SIMPLIFY` is on, choose one of:
  - Qiskit `Optimize1qGates`
  - Qiskit `Optimize1qGateDecomposition`
  - TKET `SquashTK1`
  - BQSKit `GroupSingleQuditGatePass`

So the search becomes:

$$\text{pipeline} = (\text{bundle activation}) + (\text{pass choice per active bundle})$$

That is a very natural formulation for qlego.

Even better: distinguish singleton bundles vs recipe bundles

Some bundles should allow choosing **a single pass**.

Example:

- `1Q_SIMPLIFY`
- `CLIFFORD_SIMPLIFY`

Some bundles are better treated as **micro-recipes**, not single passes.

Example:

- `LOCAL_CANCEL`
  - Qiskit `CommutationAnalysis -> CommutativeCancellation`
  - TKET `FullPeepholeOptimize`
  - BQSKit `IterativeScanningGateRemoval`
- `BLOCK_RESYNTH`
  - Qiskit `Collect2qBlocks -> ...`
  - TKET `KAKDecomposition`
  - BQSKit `QuickPartitioner -> QSearchSynthesis`

Because some passes in these bundles are helper passes and do not make sense alone.

So inside a bundle, the selectable unit should be:

- either a **single pass**
- or a **small predefined recipe**

not necessarily always one raw pass.

Best practical rule

For each bundle, define a set of **options**.

For example:

`1Q_SIMPLIFY`

- none
- `qiskit:Optimize1qGates`
- `qiskit:Optimize1qGateDecomposition`
- `tket:SquashTK1`
- `tket:SquashRzPhasedX`
- `bqskit:GroupSingleQuditGate`

`LOCAL_CANCEL`

- none
- `qiskit:CommutationAnalysis+CommutativeCancellation`
- `qiskit:CommutativeOptimization`
- `tket:FullPeepholeOptimize`
- `bqskit:ScanningGateRemoval`
- `bqskit:IterativeScanningGateRemoval`

`BLOCK_RESYNTH`

- none
- `tket:KAKDecomposition`
- `tket:PeepholeOptimize2Q`

- `bqskit:QuickPartitioner+LEAPSynthesis`
- `bqskit:QuickPartitioner+QSearchSynthesis`

This is probably the cleanest formulation.

Why this is better than selecting arbitrary raw subsets

Because otherwise the search space becomes messy:

- many passes overlap
- some passes are useless alone
- some only make sense before/after others
- attribution becomes difficult

Bundle-based choice gives you:

- interpretability
- smaller search space
- cleaner ablation
- easier cross-framework comparison

My recommendation

Yes, **within a bundle choose one candidate option**.

But let "option" mean:

- a single pass, when that makes sense
- a small pass recipe, when a helper+transformer pair is needed

So the search unit is not always a raw pass; it is a **bundle option**.

A good first version for qlego is:

- one selected option per active bundle
- fixed bundle order
- optional repetition of some bundles like `1Q_SIMPLIFY` and `LOCAL_CANCEL`

That is simple and strong.

I can next help you define the **exact option list for each of your bundles** from the passes you listed.

can you tell me about the passes that I gave you each in about 300 words

Thought for 1m 1s >

There are **36 passes** in your list, so **300 words each would become a small chapter**. The most useful way is to do this in **batches**. I'll start with the **Qiskit passes** here, with each one explained clearly and briefly. The pass catalog confirms the core role of each pass, and Qiskit also distinguishes **analysis passes** from **transformation passes**, which matters for understanding how some of these work.

Qiskit +2

Qiskit passes

1. `PresetOptimizationPass`
This is best thought of as a **macro pipeline**, not a primitive optimization pass. In Qiskit, preset pass managers are staged workflows that combine many passes into a full compilation recipe, typically including translation, layout/routing, and optimization phases. So when you use this in qlego, you are not testing one local optimization idea; you are testing a preassembled strategy. That is why I suggested

keeping it as a **baseline/comparison pipeline** rather than mixing it directly into the same search space as primitive passes. Qiskit

2. `Optimize1qGatesPass`
This is the classic single-qubit-chain compressor. Qiskit describes it as combining chains of 1-qubit gates into a single gate. Conceptually, when a circuit has long runs of adjacent 1q gates on the same wire, this pass collapses them into an equivalent shorter form. It is usually one of the safest cleanup passes to apply because it rarely depends on complicated global structure. In your bundling language, this belongs squarely in the **single-qubit simplification** bundle. Qiskit

3. `Optimize1qGateDecompositionPass`
This is similar in spirit to `Optimize1qGates`, but slightly smarter and more target-aware. Qiskit says it combines chains of single-qubit gates and decides whether to replace the original chain based on whether the chain is out of basis, whether a new synthesis has lower error, or whether the whole chain is effectively identity. That means it is not just compressing syntax; it is trying to choose a better implementation relative to a basis or target. For qlego, this is a very strong candidate for a **preferred 1q optimization option**. Qiskit

4. `Optimize1qGateSimpleCommutationPass`
This is still a 1q optimization pass, but it goes one step further: Qiskit says it optimizes 1-qubit strings that are interrupted by 2-qubit gates by commuting components and then resynthesizing. So unlike a plain adjacent-chain merge, it tries to exploit simple commutation structure to expose more 1q simplification opportunities. This makes it a good bridge between a pure **1q simplification** bundle and a **local algebraic/commutation** bundle. In practice, this is often useful after other rewrites have created partially separated 1q segments. Qiskit

5. `CommutativeCancellationPass`
This pass cancels redundant self-adjoint gates by using commutation relations. The key idea is that even when two gates are not adjacent, they may be able to move past other commuting operations until they meet and cancel. This makes it more powerful than simple back-to-back inverse cancellation. In your framework, this is a core member of the **local cancellation / commutation simplification** bundle. It is especially meaningful on circuits with repeated entangling patterns or symmetry-generated cancellations. Qiskit

6. `CommutationAnalysisPass`
This one is conceptually different from the others because it is primarily about **analysis**, not direct rewriting. Qiskit's transpiler model distinguishes analysis passes, which gather information, from transformation passes, which modify the circuit. `CommutationAnalysis` is there to identify commutation structure that later passes can exploit. So in qlego, this should usually not be benchmarked as a standalone "optimizer." It makes more sense as a **helper pass** that accompanies something like `CommutativeCancellation` or `CommutativeOptimization`. Qiskit +2

7. `CommutativeOptimizationPass`
This is the broader commutation-based optimization pass. Qiskit describes it as canceling or merging gates by exploiting commutativity relations. Compared with `CommutativeCancellation`, this sounds like a more general algebraic cleanup mechanism rather than a narrow cancellation-only step. In your experiments, I would treat this as an **alternative option** within the same commutation-oriented bundle, not as something always combined with `CommutativeCancellation`, because the two likely overlap in effect. Qiskit

8. `OptimizeCliffordsPass`
This is specialized: it combines consecutive Clifford operations acting on the same qubits. Clifford subcircuits have a lot of algebraic structure, so a generic optimizer may miss opportunities that a Clifford-aware pass can exploit efficiently. This pass therefore belongs in a **Clifford-aware simplification** bundle rather than a general local-cleanup bundle. It will matter much more on circuits with stabilizer-heavy or Clifford-dominated sections than on arbitrary parameterized ansätze. Qiskit

9. `OptimizeCliffordTPass`
This is even more specialized than `OptimizeCliffords`: it targets **Clifford+T circuits**. That makes it especially relevant if your benchmarks include fault-tolerant style cost models, T-count/T-depth

considerations, or exact-synthesis style workloads. If your workloads are mostly NISQ variational circuits over native analog rotations, this pass may not fire much. In bundling terms, I would either keep this as part of the **Clifford bundle** or split it into a narrower **Clifford+T bundle** when your benchmarks justify it. Qiskit

10. `CollectCliffordsPass`
This is a structure-exposure pass. Rather than directly optimizing, it gathers consecutive Clifford regions so that a later Clifford-aware optimizer can act on them more effectively. That means it is analogous to a "collector" or "preparation" step. In qlego, this is a classic **helper pass**, and I would normally pair it with `OptimizeCliffords` instead of testing it alone. On its own, it may expose structure without reducing cost; its value is mostly in enabling later synthesis/simplification. Qiskit

11. `Collect2qBlocksPass`
This pass collects consecutive 2-qubit blocks. The important point is that block collection is not usually the final optimization goal; it is preparing a region so that a later block-level synthesis or consolidation pass can replace that region with a better implementation. So this belongs in your **structure exposure / block formation** bundle. In experiments, it should usually be paired with some kind of 2q resynthesis choice, otherwise you are only exposing structure and not cashing in the benefit. Qiskit +1

12. `CollectMultiQBlocksPass`
This is the larger cousin of `Collect2qBlocks`: instead of restricting to 2-qubit regions, it collects multi-qubit blocks. That makes it potentially more powerful, but also potentially more expensive or more sensitive to what the downstream synthesizer can handle. This is a strong example of a pass that should almost never be treated as a standalone optimizer in your experiments. It is much better viewed as part of a **block-resynthesis workflow** where collection is followed by some synthesis or decomposition strategy. Qiskit

13. `RemoveIdentityEquivalentPass`
This is a cleanup pass that removes gates whose effect is negligible or equivalent to identity. This kind of pass is usually cheap and low-risk, and it fits naturally into a **final cleanup** bundle. It will not usually produce dramatic global improvements, but it can trim residual artifacts left by other transformations. In qlego, this is the sort of pass that can reasonably be fixed "always on" at the end, rather than consuming a lot of search budget. Qiskit

14. `OptimizeSwapBeforeMeasurePass`
This is a terminal-structure optimization. Qiskit says it removes swaps that are immediately followed by measurement, while adapting the measurement accordingly. The reason this works is that if all that matters is which classical bit gets which qubit's measured result, then sometimes the swap itself is unnecessary; you can reinterpret the measurement mapping instead. This belongs cleanly in your **measurement / terminal cleanup** bundle and is especially useful for routed circuits that end in measurements. Qiskit

15. `RemoveDiagonalGatesBeforeMeasurePass`
This is another terminal optimization. Qiskit says it removes diagonal gates, including diagonal 2-qubit gates, when they appear immediately before measurement. The intuition is that diagonal phase operations often do not affect computational-basis measurement probabilities at the end of the circuit. This pass is therefore highly context-dependent: very useful in measurement-terminated circuits, irrelevant in circuits where the state continues evolving. In qlego, it definitely belongs in the **measurement cleanup** bundle, not in a generic optimization bucket. Qiskit

16. `RemoveFinalResetPass`
This pass removes a reset when it is the last instruction on a qubit wire. That is a very targeted dead-code elimination: if nothing happens after the reset, the reset has no effect on the circuit's observable output. This is a perfect example of a cheap, semantics-preserving terminal cleanup pass. In your qlego setup, this should live either in the **measurement/terminal cleanup** bundle or in a general **final dead-code cleanup** bundle. Qiskit

A good next step is to do the **TKET passes in the same style**, then the **BQSKit passes**, and finally turn all of them into a **bundle table** with columns like: role, helper/transformer, best stage, repeatable, and whether it should be paired with another pass.

Sources